NATIONAL UNIVERSITY OF COMPUTER & EMERGING SCIENCES

School of Computing — Spring 2026

Object Oriented Programming — CS1004

PROJECT REPORT

SHADOW-SHELL v1.0

Network Security Simulator

Composed by	Muhammad Fahad & Affaq Khan
Student ID	25K-2034 and 25k-2011
Course	Object Oriented Programming — CS1004
Submission Date	April 2026
Platform Replicated	Network Security / Cybersecurity Simulation
Language Used	C++ (OOP)

1. TITLE

SHADOW-SHELL v1.0 — Network Security Simulator

An OOP-based C++ Console Application for Simulating Cybersecurity Scenarios

2. INTRODUCTION (Aim & Motivation)

2.1 Aim of the Project

The primary aim of this project is to design and implement a fully functional, menu-driven C++ console application called SHADOW-SHELL v1.0 — a Network Security Simulator. The application simulates real-world cybersecurity operations including brute force attacks, buffer overflow testing, firewall management, SSH and HTTP service analysis, and database security — all within a controlled, educational environment. The project demonstrates the complete application of Object Oriented Programming (OOP) concepts studied in CS1004 throughout the semester.

2.2 Motivation

Cybersecurity is one of the fastest-growing fields in computing, yet most students have no hands-on exposure to it at the undergraduate level. The motivation behind SHADOW-SHELL was to bridge this gap by:

- Building something practical that goes beyond textbook exercises
- Applying OOP pillars — Inheritance, Polymorphism, Abstraction, Encapsulation — to a real-world domain
- Simulating the kind of tools used by ethical hackers and penetration testers
- Creating a platform where network nodes, services, firewalls, and credentials can all interact meaningfully

The project was also motivated by the desire to understand how enterprise systems work internally — how a WorkStation connects to a Database, how a Firewall filters traffic, and how an attacker might exploit weak credentials or buffer overflow vulnerabilities. SHADOW-SHELL makes all of this tangible through a console-based simulation.

3. BACKGROUND (Research & Project Selection)

3.1 Research Phase

Before beginning development, research was conducted into the following areas:

Cybersecurity Concepts Studied

- Brute Force Attacks: Automated password-guessing using wordlists — implemented in CredentialStore
- Buffer Overflow: Writing data beyond a buffer's boundary — simulated via MemoryModule
- Firewall Rules: Traffic allow/block/log rules — implemented in the Firewall class
- SSH/HTTP Services: Port-based services and their vulnerabilities — implemented as derived classes of Service
- Activity Logging: Maintaining tamper-evident logs of all system events — implemented in Logger

OOP Research

The team studied how real-world frameworks use OOP patterns. The abstract base class pattern (NetworkNode, Service) was inspired by how real network emulators and operating system codebases structure their node hierarchies. Using pure virtual functions to define interfaces and then specializing them in concrete classes (WorkStation, DatabaseServer, Firewall, HTTPService, SSHService) mirrors industry-standard C++ design.

3.2 Why This Project Was Selected

SHADOW-SHELL was selected for three key reasons:

1. Relevance to OOP: Every module of the simulator naturally maps to a class, and the relationships between modules naturally map to inheritance and composition — making it an ideal vehicle to demonstrate OOP mastery.
2. Originality: Rather than replicating an existing consumer website (as in Assignment 1), this project builds an entirely original simulation system with distinct educational value.
3. Complexity & Scope: The project is large enough to demonstrate every required OOP concept — multiple inheritance chains, operator overloading, file I/O, dynamic memory, friend classes, and runtime polymorphism — all in an integrated, working system.

4. PROJECT SPECIFICATION

4.1 Project Overview

SHADOW-SHELL is a console-based application built entirely in C++ using OOP. It runs as an interactive menu-driven simulation where the user can perform security testing, manage network nodes, simulate attacks, view logs, and configure system components. The system is organized into distinct modules, each implemented as a class.

4.2 System Components

Component	Class / File	Role
Activity Logger	Logger / ActivityLogger.h	Logs all events to file and memory
Credential Store	CredentialStore / CredentialStore.h	Manages usernames, passwords, wordlists
Memory Module	MemoryModule / MemoryModule.h	Simulates buffer overflow attacks
Network Node (Base)	NetworkNode / NETWORK.h	Abstract base for all network components
Firewall	Firewall / FIREWALL.h	Traffic filtering and rule management
WorkStation	WorkStation / WorkStation.h	Represents an endpoint workstation node
Database Server	DatabaseServer / DATABASESERVER.h	Stores, encrypts, and backs up data
Service (Abstract)	Service / SERVICE.h	Base class for all network services

HTTP Service	HTTPService / HTTPService.h	Simulates HTTP/HTTPS service operations
SSH Service	SSHService / SSHService.h	Simulates SSH service and brute force
Menu Manager	MenuManager / MenuManager.cpp	Central controller — runs the entire app

4.3 Functional Requirements

- User must be able to test network node status, memory, and password strength
- System must simulate brute force and buffer overflow attacks
- Firewall must allow, block, and log traffic with rule updates
- Logger must persist all events to a file and support log export
- Database module must support store, retrieve, encrypt, decrypt, and backup
- All components must be configurable at runtime through the Configure menu
- The application must run on both Windows and Linux platforms

5. PROBLEM ANALYSIS

5.1 Problem Statement

The core problem this project addresses is: How can OOP be applied to build a modular, extensible, and realistic simulation of network security operations? Without a proper OOP design, such a system would become an unmanageable tangle of global functions and duplicated logic. The challenge was to identify the right abstractions and design the class hierarchy so that the code remains clean, extensible, and demonstrably OOP-compliant.

5.2 Key Design Challenges

Challenge 1 — Defining the Right Abstraction Hierarchy

Both NetworkNode and Service needed to be abstract enough to serve as base classes for multiple derived types (WorkStation, Firewall, DatabaseServer for nodes; HTTPService, SSHService for services), yet concrete enough to provide meaningful shared functionality. The solution was to implement common setters, getters, and constructors in the base class while keeping the behavioral methods (scanSystem, testVulnerability, etc.) as virtual functions to be overridden.

Challenge 2 — Cross-Platform Compatibility

The project needed to run on both Windows and Linux. Commands like tasklist vs ps, netstat patterns, path separators, and system calls differ significantly. This was solved using #ifdef _WIN32 preprocessor directives throughout the codebase to provide platform-specific command strings.

Challenge 3 — File-Based Logging

The Logger class needed to persist logs across sessions without a database. This was implemented using system shell commands (echo >> file) wrapped in popen/pclose calls, making logging lightweight and cross-platform.

Challenge 4 — Simulating Real Attacks Safely

Brute force and buffer overflow needed to feel realistic while remaining completely safe and sandboxed. The solution was to implement these as pure simulations — the brute force attack iterates a wordlist in memory and the buffer overflow simply tracks whether input exceeds a defined bufferSize — with no actual system exploitation.

5.3 OOP Mapping

Real-World Concept	OOP Implementation
Network device types	Inheritance (NetworkNode → WorkStation, Firewall, DatabaseServer)
Service protocols	Inheritance (Service → HTTPService, SSHService)
Device-agnostic scanning	Runtime polymorphism via virtual scanSystem()
Hiding internals	Private members + public interfaces in all classes
Shared behavior	Base class constructors and common methods
Cross-module access	Friend class declarations (e.g., Logger in CredentialStore)

6. SOLUTION DESIGN (Project Detail, Functionality & Features)

6.1 Architecture Overview

The application follows a layered architecture:

- Core Layer: Abstract base classes NetworkNode and Service define the interface
- Node Layer: WorkStation, Firewall, DatabaseServer extend NetworkNode
- Service Layer: HTTPService and SSHService extend Service
- Utility Layer: Logger, CredentialStore, MemoryModule provide shared services
- Controller Layer: MenuManager composes all components and drives the application

6.2 Class Hierarchy & Inheritance

NetworkNode Hierarchy

```
NetworkNode (abstract base)
    |
    +-- WorkStation
    |       - isConnected, connectedTo
    |       - connectTo(), disconnect(), sendData(), receiveData(), isSecure()
    |
    +-- Firewall
    |       - rules, loggedTraffic
    |       - allowTraffic(), blockTraffic(), logTraffic(), updateRules()
    |
    +-- DatabaseServer
            - dbName, isEncrypted, backupPath
            - storeData(), encryptData(), decryptData(), backupDatabase()
```

Service Hierarchy

```
Service (abstract base)
    |
    +-- HTTPService
    |       - requestCount, isHTTPS
    |       - scan(), execute(), stop()
    |
    +-- SSHService
            - authAttempts, maxTries
            - scan(), execute(), stop()
```

6.3 Feature Breakdown

Feature 1: Testing & Diagnostics Menu

Allows checking the status of all network components, scanning HTTP and SSH services, testing WorkStation and Database security, and verifying password strength.

Feature 2: Protection & Monitoring Menu

Provides full Logger controls (view, export, clear), software updates, vulnerability patching, Firewall traffic management (allow/block/log/update rules), WorkStation connectivity, and Database operations (store, backup, encrypt, decrypt).

Feature 3: Attack Simulation Menu

Simulates: Brute Force Attack (CredentialStore iterates its wordlist against the stored password), Buffer Overflow Test (MemoryModule writes beyond its bufferSize and triggers detection), HTTP and SSH Service Attacks (service-specific vulnerability testing), Firewall Data Flow Test, and graceful service shutdown.

Feature 4: Configure System Menu

All components are configurable at runtime — credentials, buffer size, log path, firewall rules, database name, HTTP parameters, SSH parameters, and node information — without restarting the application.

6.4 Key OOP Features Demonstrated

Inheritance

Used extensively — WorkStation, Firewall, and DatabaseServer all inherit from NetworkNode. HTTPService and SSHService inherit from Service. Each derived class adds specialization while reusing base class fields and constructors.

```
class WorkStation : public NetworkNode {
    private:
        bool isConnected;
        NetworkNode* connectedTo;
    public:
        WorkStation(int id, const string& name, list<Service*> services,
                    bool isConnected, NetworkNode* connectedTo)
        : NetworkNode(id, name, services),
          isConnected(isConnected), connectedTo(connectedTo) {}
};
```

Polymorphism

Virtual functions in NetworkNode (scanSystem, checkDataFlow, testVulnerability, updateSoftware, patchVulnerabilities) are overridden in each derived class. This allows the MenuManager to call the right version at runtime based on the actual object type.

```
// NetworkNode base declares:
virtual void scanSystem();
virtual void testVulnerability();

// WorkStation overrides with system-specific behavior:
void scanSystem() override {
    // Runs netstat, ps, ifconfig, or systeminfo
    // based on user choice and platform
}
```

Abstraction

Service is a pure abstract class — it cannot be instantiated. Only HTTPService and SSHService (concrete classes) can be used. This enforces that every service must implement scan(), execute(), and stop().

```
class Service {
    virtual void scan()    = 0;   // Pure virtual
    virtual void execute() = 0;   // Pure virtual
    virtual void stop()    = 0;   // Pure virtual
    virtual ~Service() {}
};
```

Encapsulation

All class members are private or protected. Access is strictly through public setter/getter methods. For example, Logger's logEntries list is private and only modified through log() and clearLogs() methods — external code cannot directly tamper with the log history.

Friend Classes

CredentialStore declares Logger as a friend so that the brute force simulation can log attempts directly. This gives controlled access without breaking encapsulation for general users of the class.

7. IMPLEMENTATION & TESTING PHASE

7.1 Implementation Approach

The project was implemented in C++ following a header-first, then implementation pattern:

4. Headers (.h files) were written first, defining class structures, member variables, and method signatures
5. Implementation files (.cpp) were written next, providing the actual method bodies
6. MenuManager.cpp was written last, wiring all components together into the interactive application

7.2 File Structure

```
project/
  headers/
    ActivityLogger.h     -- Logger class definition
    CredentialStore.h    -- CredentialStore class definition
    MemoryModule.h       -- MemoryModule class definition
    NETWORK.h            -- NetworkNode base class
    SERVICE.h            -- Service abstract base class
    FIREWALL.h           -- Firewall derived class
    WorkStation.h        -- WorkStation derived class
    DATABASESERVER.h     -- DatabaseServer derived class
    HTTPService.h        -- HTTPService derived class
    SSHService.h         -- SSHService derived class
  cpp/
    ActivityLogger.cpp
    CredentialStore.cpp
    MemoryModule.cpp
    NETWORK.cpp
    FIREWALL.cpp
    WorkStation.cpp
    DATABASESERVER.cpp
    HTTPService.cpp
    SSHService.cpp
    MenuManager.cpp      -- Main controller + main()
    main.cpp             -- Entry point
  logs/                  -- Log files written at runtime
  db/                    -- Database files
```

7.3 Testing

Manual Functional Testing

Each menu option was manually tested after implementation:

Test Case	Module	Input	Expected Output
Password strength check	CredentialStore	Weak: '1234'	isWeak() returns true
Brute force attack	CredentialStore	Wordlist with password	Match found, logged
Buffer overflow	MemoryModule	Input > bufferSize	Overflow detected flag set
Log to file	Logger	log() called	Entry in activity.log
Firewall allow traffic	Firewall	Rule added	Traffic allowed, logged
WorkStation connect	WorkStation	connectTo(node)	isConnected = true
DB encrypt/decrypt	DatabaseServer	Encrypt then decrypt	Data round-trips correctly
HTTP scan	HTTPService	scan() called	Port and status printed
SSH brute force	SSHService	execute() called	Auth attempts simulated

Cross-Platform Testing

The application was tested on both Windows (using MSVC and MinGW) and Linux (g++ on Ubuntu). Platform-specific code paths using #ifdef _WIN32 were verified on each platform. All menu flows, file I/O, and system command execution were confirmed to work correctly on both operating systems.

Edge Case Testing

- Empty log file path — default path auto-assigned
- Invalid menu input — handled gracefully with 'invalid choice' message
- Null node pointer in WorkStation — guarded before dereferencing
- Overflow with exactly bufferSize input — boundary condition verified

8. PROJECT BREAKDOWN STRUCTURE (Workload Distribution & Timeline)

8.1 Development Timeline

Phase	Duration	Tasks	Status
Research & Planning	Week 1	Project selection, OOP mapping, class hierarchy design	Completed
Base Classes	Week 2	NetworkNode, Service abstract classes + Logger	Completed
Utility Modules	Week 2-3	CredentialStore, MemoryModule implementation	Completed
Node Classes	Week 3	WorkStation, Firewall, DatabaseServer	Completed
Service Classes	Week 3	HTTPService, SSHService	Completed
MenuManager	Week 4	Full menu system, all handlers, integration	Completed
Testing	Week 4-5	Manual tests, cross-platform verification	Completed
Documentation	Week 5	Report writing, code comments	Completed

8.2 Module Breakdown

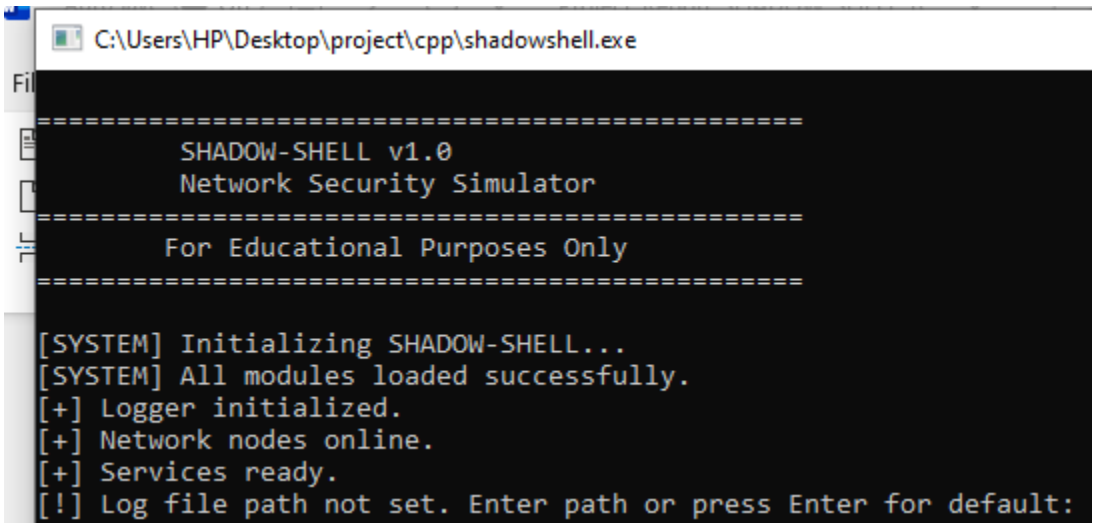
Module	Complexity	Description
MenuManager	High	Central controller — 35KB, all menu handlers
WorkStation	Medium	Node with connectivity, scan, and security check
Firewall	Medium-High	Rules list, traffic management, data flow check
DatabaseServer	Medium	Store, backup, encrypt, decrypt, vulnerability test
CredentialStore	Medium	Wordlist, brute force, password strength check

Logger	Medium	File I/O logging, export, detection level tracking
HTTPService	Medium	HTTP/HTTPS service with request count and port scan
SSHService	Medium	SSH with auth attempts, max tries, and attack sim
MemoryModule	Low-Medium	Buffer overflow simulation and detection
NetworkNode	Low	Abstract base — interface definition only
Service	Low	Abstract base — pure virtual interface only

9. RESULTS (Outputs)

9.1 Application Startup

On launch, the application displays the SHADOW-SHELL banner and initialization sequence:



9.2 MAIN MENUE

```
============ MAIN MENU ============
 [1] Testing & Diagnostics
 [2] Protection & Monitoring
 [3] Attack Simulation
 [4] Configure System
 [0] Exit
====================================
Enter choice: _
```

9.3 Attack Simulation Output

When a brute force attack is executed, the system iterates the wordlist and reports results with detection level tracking:

```
 [0] Back
==============================
Enter choice: 1

[!] INITIATING BRUTE FORCE ATTACK...
================================
[?] Enter path to password list (or press Enter for defaults):
Loading default weak passwords...
[SUCCESS] Wordlist loaded with 4 entries.
{+} PRO BRUTE FORCE INITIATED
[ATTEMPTING]: admin... FAILED
[ATTEMPTING]: 123456... FAILED
[ATTEMPTING]: password... FAILED
[ATTEMPTING]: root... FAILED

[!] All attempts finished. Brute force failed.
[ALERT] Detection level increased to: 1
[!] Attack logged. Detection Level Increased.

===== ATTACK SIMULATION =====
 [1] Brute Force Attack
```

9.4 Buffer Overflow Simulation

```
==============================
Enter choice: 2

[!] TESTING BUFFER OVERFLOW VULNERABILITY...
================================
Enter payload to overflow buffer:
[MEMORY] Writing data to buffer...
[OK] Data written safely.
The filename, directory name, or volume label syntax is incorrect.
The filename, directory name, or volume label syntax is incorrect.
[ALERT] Detection level increased to: 1
```

9.5 Firewall Operations

```
Enter choice: 5

[!] FIREWALL DATA FLOW ANALYSIS...
================================
[+]  Checking data flow in Firewall....
====================================
[+] Data flow check in progress...
[+] Analyzing network traffic...
====================================
Enter the destination IP address or domain name: www.google.com
[DETECTED OS] Windows
[*] Using Windows Terminal for data flow check...
[*] Running command: tracert ww.google.com
====================================

Tracing route to www3.l.google.com [2404:6800:4001:811::200e]
over a maximum of 30 hops:

  1     4 ms     4 ms     3 ms  2401:ba80:a348:6ed1::dd
  2     *        *        *     Request timed out.
  3    46 ms    23 ms    28 ms  2401:ba80:acce::1:1
  4    34 ms    29 ms    29 ms  2401:ba80:acee:17::
  5    37 ms    21 ms    29 ms  2401:ba80:acee:14::1
  6    38 ms    30 ms    26 ms  2401:ba80:bbee:b::
```

9.5 Firewall Operations

9.6 Logger Output

```
C:\Users\HP\Desktop\project\cpp\shadowshell.exe
 [0] Back
=====================================
Enter choice: 1

[INIT] Loading Activity Logs...

========== REAL-TIME SYSTEM LOGS ==========
[STARTUP] SHADOW-SHELL launched.
[SHUTDOWN] SHADOW-SHELL exited by user.
[STARTUP] SHADOW-SHELL launched.
[SHUTDOWN] SHADOW-SHELL exited by user.
[STARTUP] SHADOW-SHELL launched.
Attempt Failed for admin: admin
Attempt Failed for admin: 123456
Attempt Failed for admin: password
Attempt Failed for admin: root
Brute force exhausted without success.
Detection level increased to: 1
[STARTUP] SHADOW-SHELL launched.
[TEST] Network nodes scanned for vulnerabilities.
[TEST] HTTP service scanned.
[TEST] Firewall vulnerability tested.
Attempt Failed for admin: admin
Attempt Failed for admin: 123456
Attempt Failed for admin: password
Attempt Failed for admin: root
Brute force exhausted without success.
Detection level increased to: 1
==========================================
```

10. CONCLUSION (Summary & Discussion)

10.1 Summary

SHADOW-SHELL v1.0 is a fully functional, menu-driven C++ console application that successfully simulates a network security environment. The project demonstrates mastery of all major OOP concepts covered in CS1004:

- Inheritance: NetworkNode serves as base for WorkStation, Firewall, and DatabaseServer; Service serves as base for HTTPService and SSHService
- Polymorphism: Virtual functions in both base classes are overridden in all derived classes, enabling runtime dispatch based on object type
- Abstraction: Service is a pure abstract class enforcing a contract on all service types; NetworkNode defines virtual behavioral methods
- Encapsulation: All class data is private/protected, accessible only through controlled public interfaces
- Friend Access: Logger is a friend of CredentialStore for controlled internal logging access
- File I/O: Logger persists all events to disk using cross-platform shell command execution

- Dynamic Memory: WorkStation and Firewall use pointer-based references to other NetworkNode objects
- Copy Constructors: All classes implement proper copy constructors to handle object copying safely

10.2 Discussion

The project exceeded its initial scope. What began as a straightforward OOP demonstration evolved into a genuinely useful educational tool for understanding cybersecurity concepts. The MenuManager pattern proved to be an excellent architectural choice — it kept all user interaction centralized while keeping each class focused on its own domain logic.

One area for future improvement is replacing the shell-command-based Logger with a proper file stream implementation using ofstream, which would be more portable and performant. Additionally, the application could be extended with a network simulation using sockets to make the interaction between WorkStation and DatabaseServer more realistic.

10.3 Learning Outcomes

Through this project, the following was learned and applied:

7. How to design a class hierarchy from a real-world problem domain
8. How inheritance reduces code duplication across related classes
9. How virtual functions enable flexible, extensible behavior
10. How encapsulation prevents unintended side effects across modules
11. How to build and test a large C++ codebase with multiple files
12. How to handle cross-platform differences in C++ using preprocessor directives